CoxyInsure DeFi Insurance DApp

Back End Systems Interaction Design

Working design document for implementation, review, and operational governance

Document Type	Back End Design / System Interactions
Purpose	Describe server-side architecture, actors, components, and operational flows
Version	1.0
Status	Draft for implementation and review
Date	2026-04-15

Prepared for the CoxyInsure protocol using the same back-end interaction structure and dark-node diagram style as the supplied reference guide.

Table of Contents

1. Overview

2. Design Goals

3. Actors and Responsibilities

4. Back End Architecture Overview

5. Core Back End Components

6. Authentication, Session, and Wallet Binding

7. Insurance Lifecycle Interactions

    7.1 Membership and wallet approval

    7.2 Purchase cover

    7.3 Submit claim

    7.4 Governance voting and admin review

    7.5 Execute payout or reject claim

8. Transaction Logging and Reporting

9. Security Controls and Trust Boundaries

10. Operational Flows and Interaction Diagrams

11. Data Storage and Persistence

12. API and Integration Endpoints

13. Error Handling and Recovery

14. Diagram Inventory

15. Summary

1. Overview

The CoxyInsure backend coordinates authenticated user access, wallet binding, cover purchase persistence, claim evidence storage, governance review, payout control, and operational reporting. Its role is to keep sensitive credentials off the client, enforce application rules, and maintain a reliable bridge between the web interface, MySQL records, and Cardano infrastructure.

The design follows a hybrid model. Critical asset movement settles through Lucid-driven Cardano transactions and validator logic, while fast dashboards, claim evidence, admin decisions, and analytics remain off-chain for reliable rendering and review.

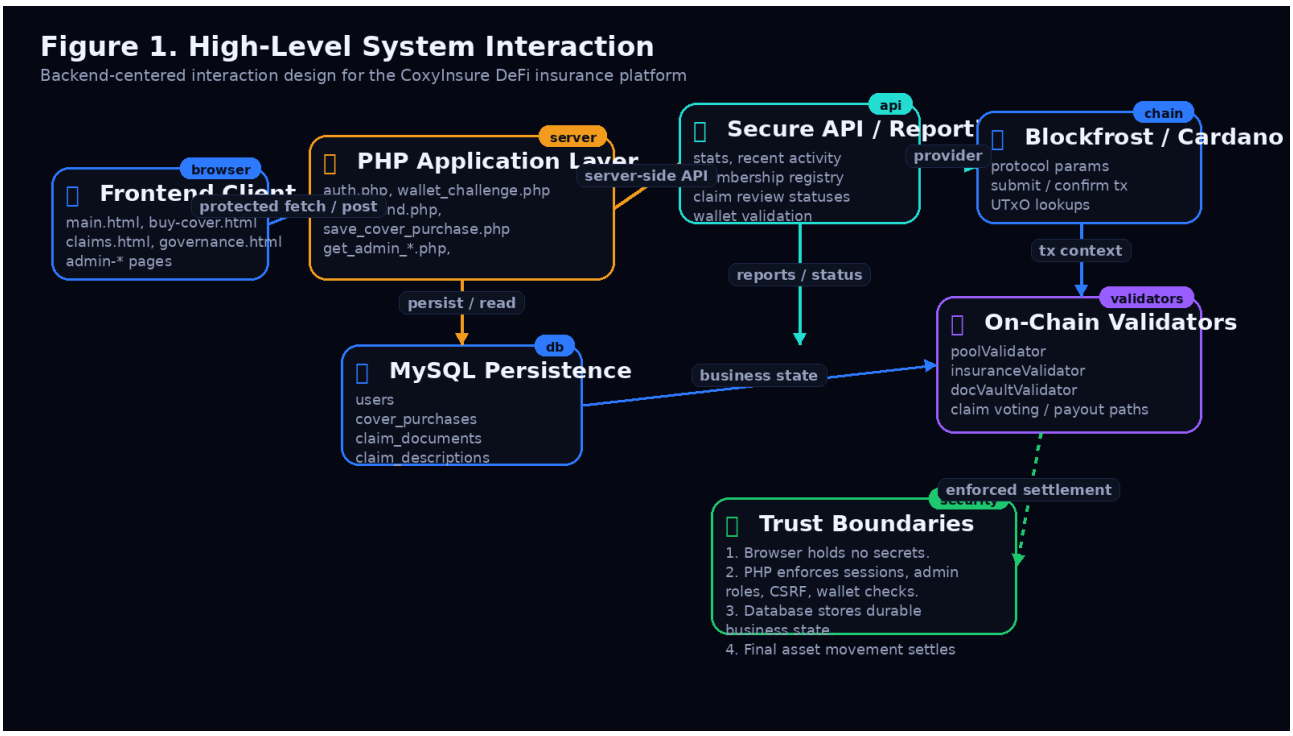

Figure 1. High-Level System Interaction

2. Design Goals

- Keep sensitive keys, session logic, and admin authorization server-side.
- Enforce authenticated sessions and wallet-to-account binding before protected user actions are allowed.
- Persist cover purchases, claim documents, descriptions, and transaction logs for fast rendering and auditability.
- Support clear lifecycle transitions across membership, cover purchase, claim submission, governance review, and payout execution.
- Provide administrator-facing oversight with separate login, role gates, CSRF checks, and persistent review outcomes.

3. Actors and Responsibilities

Actor	Primary Back-End Interaction	Typical Permissions

Member / Liquidity Provider	Connects wallet, binds wallet, mints membership, deposits ADA, withdraws shares if no active cover lock applies	Mint membership, deposit to pool, view own activity, withdraw when protocol rules permit
Cover Buyer	Connects wallet, purchases cover, pays premium, appears in active-cover registry	Buy cover, view own covers, submit claim evidence, check cover deadlines
Claimant	Uploads incident document, stores description, submits on-chain claim transaction	Create own claim, view own claim status, participate in governance when eligible
Governance Participant	Votes on pending claims through the public governance portal	Cast votes, view threshold status, inspect evidence
Admin / Reviewer	Uses separate admin login, views covers, claims, members, and governance queue	Reject claim, execute approved claim, review logs, run oversight searches
Application Back End	Validates sessions, binds wallets, stores evidence, logs business events, serves dashboard APIs	Server-side enforcement and orchestration

4. Back End Architecture Overview

The backend uses a layered structure. The browser interacts with protected PHP endpoints. Those endpoints enforce login, wallet ownership, admin roles, CSRF checks, and input validation; store or retrieve records from MySQL; and invoke Cardano provider infrastructure only through controlled server-side routes or browser wallet transactions.

- Presentation boundary: browser UI, wallet prompts, user and admin dashboards.
- Application boundary: PHP endpoints for auth, wallet binding, cover purchase save, claims, review state, stats, and reporting.
- Persistence boundary: MySQL tables for users, covers, claims, documents, admin reviews, and transaction logs.
- Blockchain boundary: Lucid transactions in the browser plus validator-backed Cardano state and Blockfrost support.
- Trust boundary: administrator actions require separate session state from user wallet-bound flows.

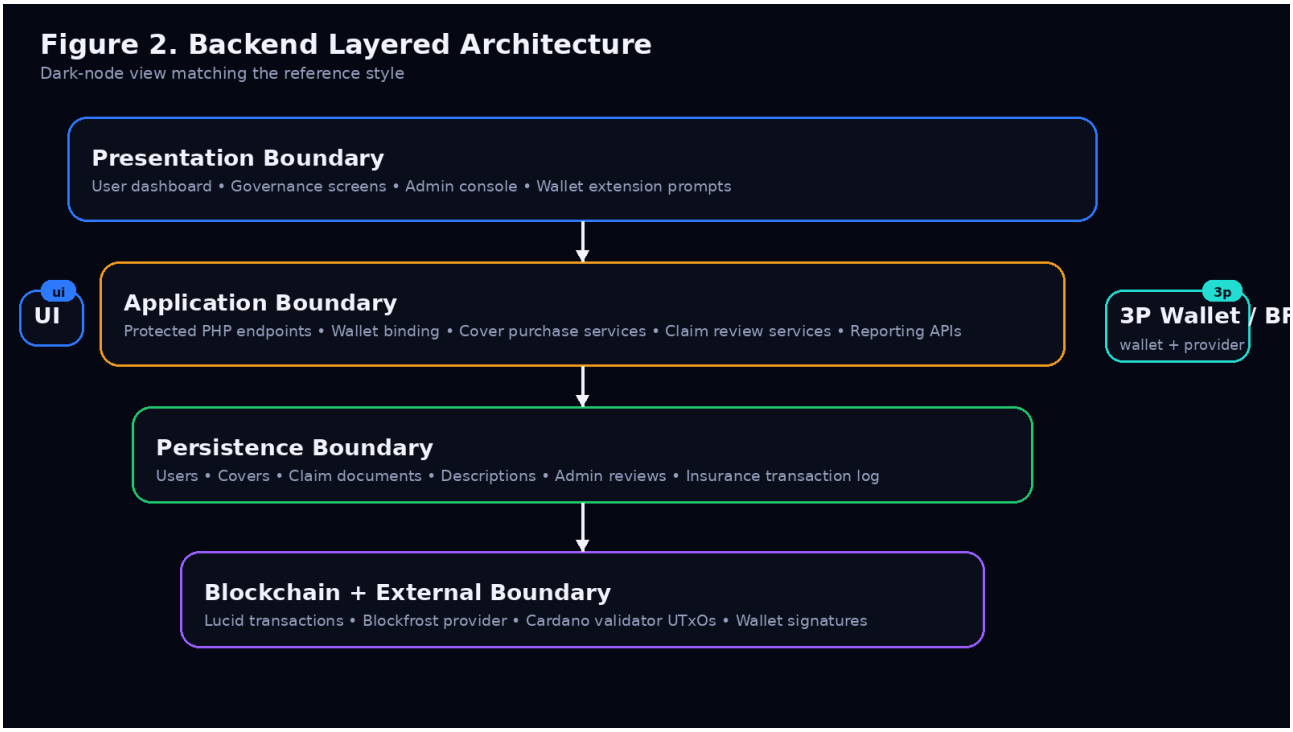

Figure 2. Backend Layered Architecture

5. Core Back End Components

Authentication and Session Layer

Handles login state, protected routes, session initialization, admin session checks, and CSRF-related controls.

Wallet Binding Services

Issues nonce-based wallet challenges, verifies signing state, and links the approved wallet address to the authenticated user account.

Cover Purchase Persistence

Stores wallet address, premium paid, selected protocol, coverage amount, duration, status, and expiry in cover_purchases.

Claim Evidence Services

Stores uploaded documents, deterministic hashes, document URLs, and claim descriptions linked by asset_unit.

Review and Governance Services

Stores admin review outcomes, claim review statuses, and the visibility state used by governance screens.

Transaction Logging Layer

Writes deposit, claim submission, vote, execute, and membership events into insurance_transactions for reporting.

Reporting Services

Generates recent activity, dashboard statistics, wallet-based searches, active-cover views, and admin monitoring datasets.

6. Authentication, Session, and Wallet Binding

Before any sensitive user action is performed, the backend requires a valid authenticated session. Wallet binding then ensures that the connected wallet is the wallet approved for that user account. Admin pages deliberately bypass user wallet binding and instead rely on a separate admin login plus optional wallet connection for execution actions only.

- User signs in through the standard application login flow.
- Backend checks whether a verified wallet already exists for the account through wallet_status.php.
- If needed, wallet_challenge.php issues a nonce-based challenge message.
- Frontend asks the wallet to sign the challenge through signData().
- wallet_bind.php stores the verified wallet address and marks it as approved for the session user.
- Subsequent user-side requests compare the connected wallet against the approved wallet record.
- Admin pages use admin_login.php, admin_auth.php, admin_security.php, role checks, and CSRF tokens instead of user wallet binding.

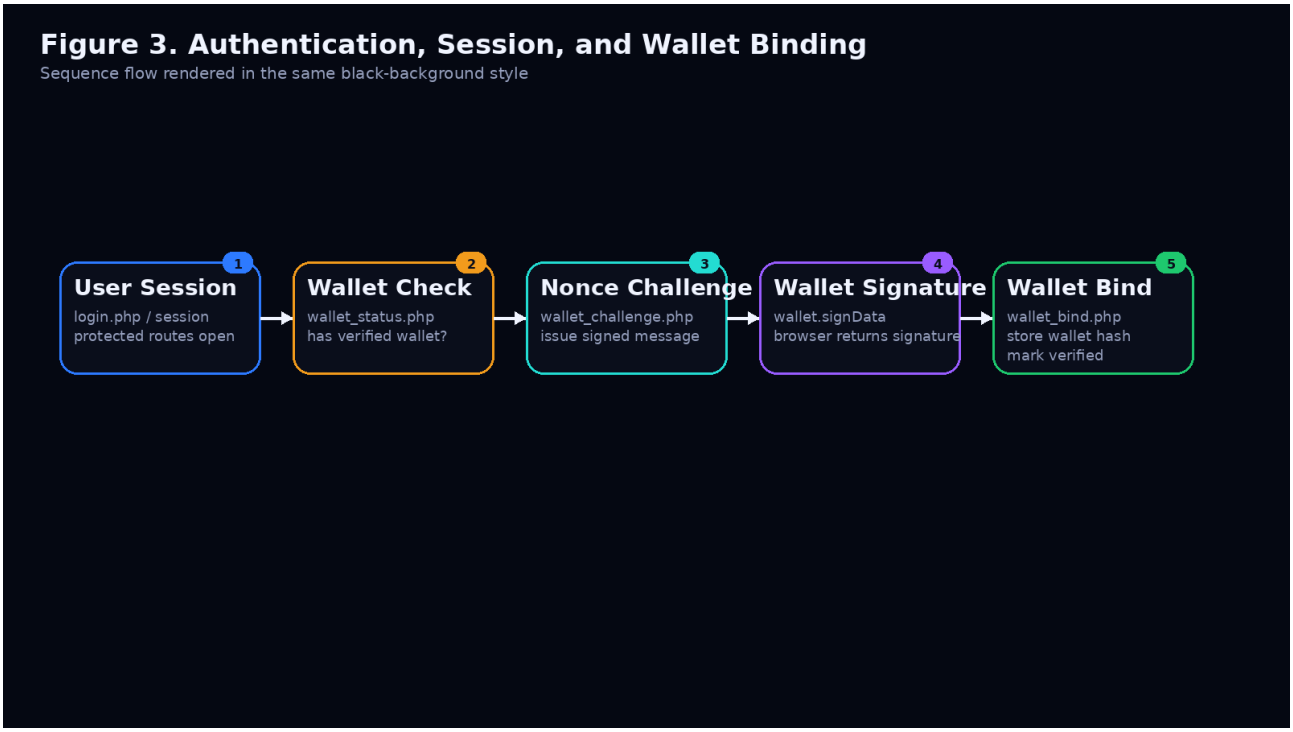

Figure 3. Wallet Binding Flow

7. Insurance Lifecycle Interactions

The insurance lifecycle is the central business flow supported by the backend. Each stage produces both on-chain effects and off-chain persistence updates.

7.1 Membership and wallet approval

Users establish a normal application session, connect the expected wallet, complete wallet binding, and then mint the membership SBT. Membership acts as a prerequisite for protected protocol participation.

7.2 Purchase cover

Users select a protocol, fixed duration, and a predefined coverage tier. The frontend calculates a fixed premium from the chosen tier, executes depositToPool(), and then save_cover_purchase.php records the off-chain cover state.

7.3 Submit claim

Users upload supporting evidence, receive deterministic document metadata, submit the claim on-chain, and then persist descriptions and document links keyed by asset_unit.

7.4 Governance voting and admin review

Public governance screens render pending claims and executable claims. Admin review status is fetched from backend review maps so rejected or executed claims are hidden from public execution queues.

7.5 Execute payout or reject claim

Admin governance connects a wallet for chain execution, but administrative authority still comes from admin session controls. reject_claim.php updates review state immediately; mark_claim_executed.php records successful execution outcomes and ties them back to tx hashes.

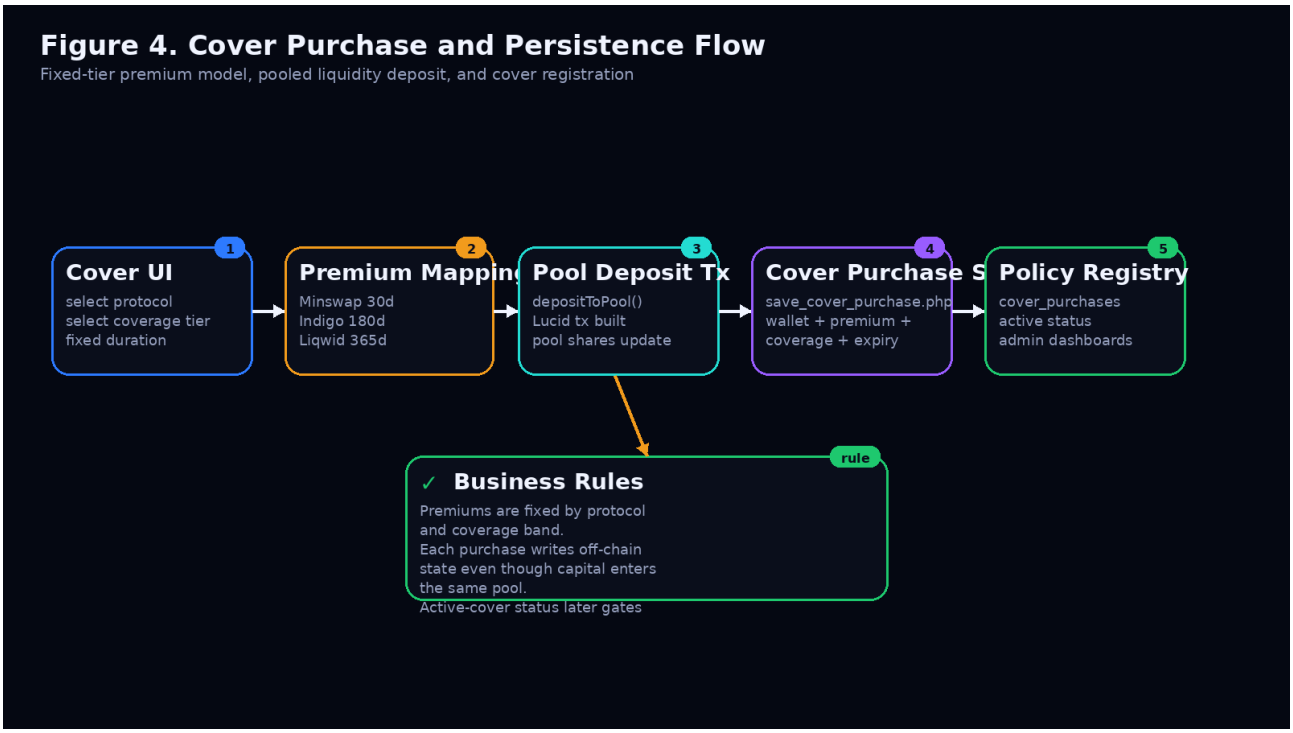

Figure 4. Cover Purchase and Persistence Flow

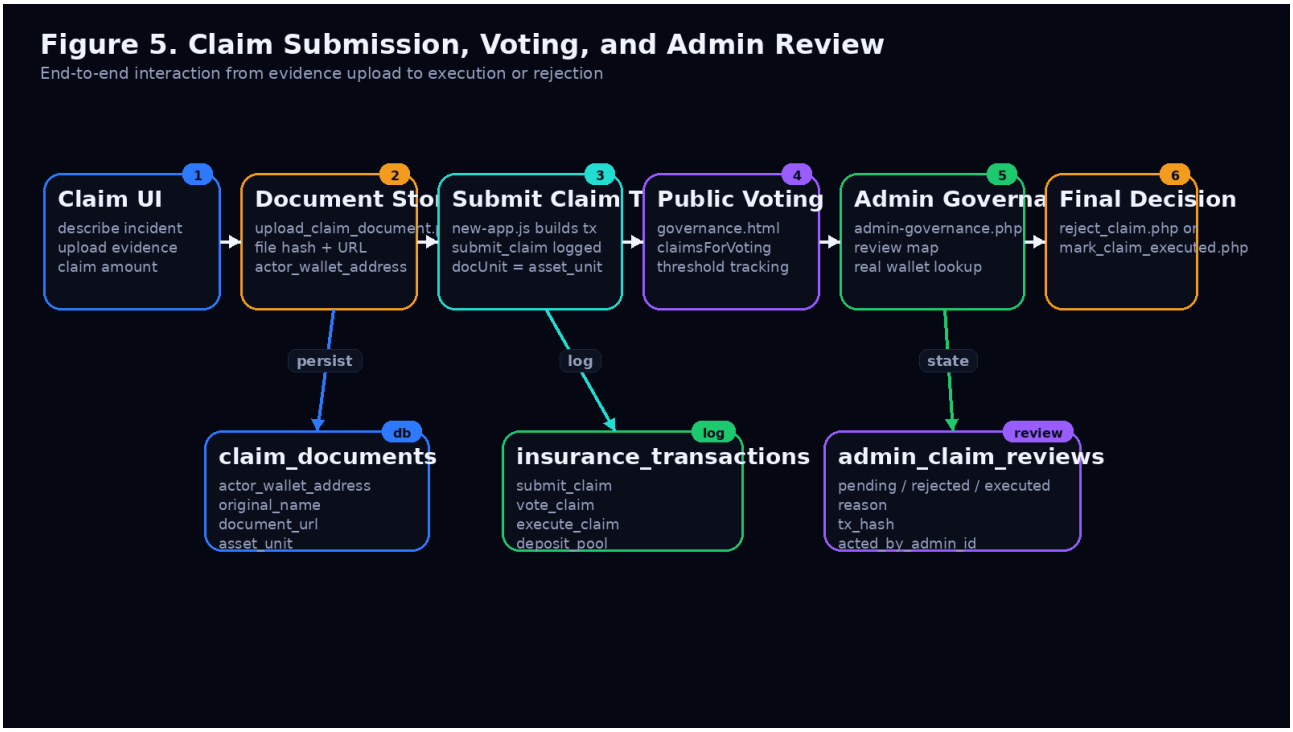

Figure 5. Claim Submission, Voting, and Admin Review

8. Transaction Logging and Reporting

The backend transaction log is essential because blockchain confirmation alone is not sufficient for fast operational views. Application-level records make it possible to populate recent activity, admin dashboards, wallet history search, and later CSV-style exports efficiently.

- Each deposit_pool, submit_claim, vote_claim, execute_claim, withdraw_shares, and mint_membership_sbt action writes a transaction row with actor wallet, amount, status, and reference identifiers.
- Recent activity panels read from insurance_transactions for dashboard widgets and history pages.
- Claim review maps read from admin_claim_reviews so governance views can exclude rejected or already executed claims.
- Membership and admin registry pages read from users and cover_purchases for user-level oversight.

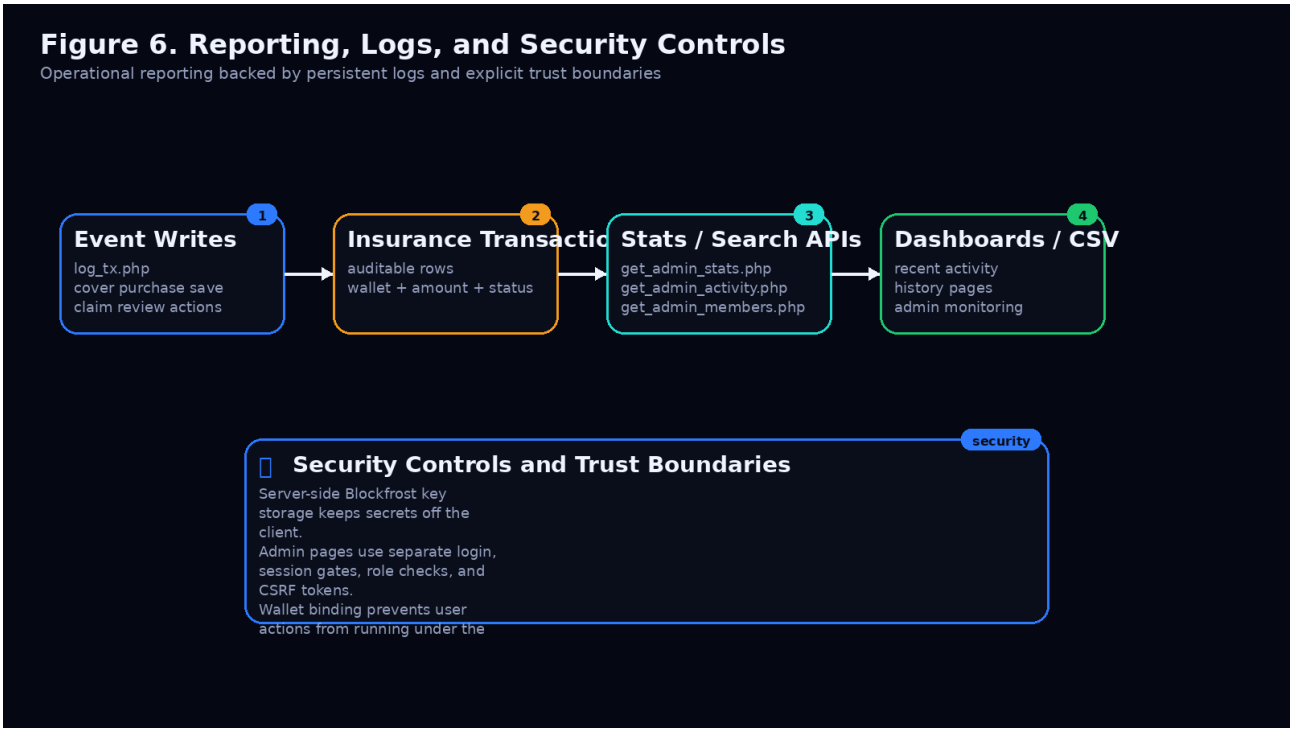

Figure 6. Reporting, Logs, and Security Controls

9. Security Controls and Trust Boundaries

Control	Risk Addressed	Back-End Response
Server-side provider configuration	API key exposure in the browser	Provider secrets remain in environment configuration and are used only by server-side routes or trusted browser wallet initialization logic.
Wallet binding	Wrong user operating another wallet	Backend compares the connected wallet against the approved wallet record before allowing sensitive user actions.
Admin login separation	User session misused for admin operations	Admin routes require admin session state and do not reuse normal user binding flow.
Admin CSRF tokens	Cross-site request forgery on execute/reject actions	Mutation endpoints require X-Admin-CSRF and active admin sessions.
Protected endpoints	Unauthenticated direct access	require_login() and require_admin() style gates protect all sensitive PHP routes.
Input validation	Malformed wallet, cover, or	Endpoints trim, validate, reject

	claim data	missing fields, and sanitize file uploads.
Transaction logging	Weak audit trail	Persistent rows record deposits, claims, votes, executions, membership events, and related reference IDs.

10. Operational Flows and Interaction Diagrams

The operating model groups backend work into authentication, membership enablement, cover purchase, claim submission, governance review, payout execution, and reporting. Each flow terminates in a persisted state so dashboards and admin pages can render without expensive chain scans.

- Authentication flow: login, session creation, wallet status check, optional wallet bind.
- Protocol access flow: membership mint and eligibility checks.
- Cover purchase flow: select tier, compute premium, deposit to pool, store cover row, update active-cover views.
- Claim flow: upload evidence, submit claim, persist document and description, log transaction.
- Governance flow: load voting queue, fetch review map, hide rejected or executed claims, allow public votes.
- Admin flow: connect wallet for execution, reject or execute claim, mark outcome and tx hash, refresh visibility.
- Reporting flow: fetch recent activity, active covers, members, and dashboard statistics.

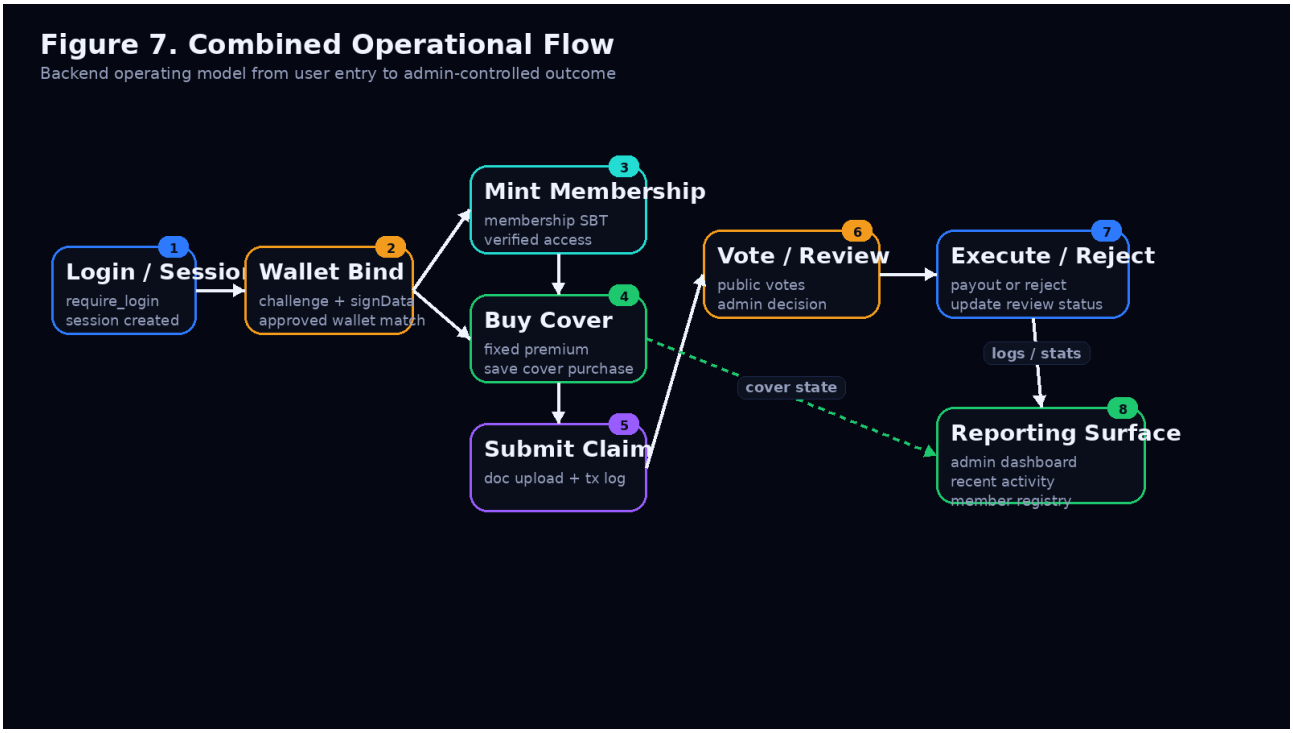

Figure 7. Combined Operational Flow

11. Data Storage and Persistence

The backend stores durable business data off-chain while keeping transaction-critical state on-chain. This hybrid model improves performance, simplifies UI rendering, and reduces reliance on repeated provider fetches.

- On-chain: pool state, claim-related validator state, payout settlement paths, membership-related transaction outputs.
- Off-chain in MySQL: users, cover_purchases, claim_documents, claim_descriptions, insurance_transactions, admin_claim_reviews, and admin accounts.
- Session state: authenticated user identity, wallet challenge nonce, admin session identity, and CSRF tokens.
- Derived rendering state: active cover counts, wallet-based search results, review maps, recent activity, and dashboard statistics.

12. API and Integration Endpoints

Capability	Representative Endpoints / Components
Authentication / user session	login.php, logout.php, auth.php
Wallet binding	wallet_status.php, wallet_challenge.php, wallet_bind.php
Cover purchase / cover state	save_cover_purchase.php, get_active_covers.php, get_admin_covers.php, get_all_admin_covers.php
Claims and evidence	upload_claim_document.php, claim_document_lookup.php, download_claim_document.php, save_claim_description.php, get_admin_claims.php, get_recent_cover_claims.php
Governance and review	get_claim_review_statuses.php, reject_claim.php, mark_claim_executed.php
Admin auth and security	admin_login.php, admin_logout.php, admin_auth.php, admin_security.php, admin_session_info.php
Reporting and statistics	get_admin_stats.php, get_admin_activity.php, get_admin_members.php, stats.php, log_tx.php
Chain interaction support	Lucid wallet methods in new-app.js, provider routes, Blockfrost-backed lookups

13. Error Handling and Recovery

- Authentication failures redirect the user back to login or return session-expired responses for protected API calls.
- Wallet mismatch blocks blockchain actions until the correct approved wallet is connected.
- Admin pages skip user wallet binding but still fail closed if admin session or CSRF validation is missing.
- Upload failures must not create partial claim-document records; duplicate file hashes should be handled safely.
- Transaction submission failures are surfaced to the UI and must not be silently marked successful in logs.
- Governance and admin pages should degrade gracefully by showing empty-state messages when no records are available.
- Rejected and executed claims are removed from executable queues through backend review status filtering.

14. Diagram Inventory

Figure	Purpose
Figure 1. High-Level System Interaction	Shows the major backend, database, API, and validator relationships.
Figure 2. Backend Layered Architecture	Separates presentation, application, persistence, and blockchain boundaries.
Figure 3. Wallet Binding Flow	Illustrates user session, challenge issue, wallet signature, and bind success.
Figure 4. Cover Purchase and Persistence Flow	Shows fixed-tier premium selection, pool deposit, and cover registration.
Figure 5. Claim Submission, Voting, and Admin Review	Connects claim evidence, claim tx, public voting, admin review, and review persistence.
Figure 6. Reporting, Logs, and Security Controls	Explains how logs feed stats and where trust boundaries sit.
Figure 7. Combined Operational Flow	Summarizes the end-to-end backend operating model.

15. Summary

The CoxyInsure backend is designed as a controlled interaction layer between authenticated users, bound wallets, persistent cover and claim records, admin oversight flows, and Cardano infrastructure. The supplied backend interaction template has therefore been adapted into a DeFi insurance operating model that supports membership gating, premium-backed cover purchase, evidence-backed claims, governance voting, admin execution or rejection, and durable reporting.

The resulting system is intentionally hybrid: validators and Lucid transactions protect asset movement and claim settlement, while the PHP and MySQL layers provide accountability, fast rendering, review state, and operator visibility.